NCCLPol: Verified, Composable Policy Execution for GPU Collective Communication

Anonymous Author(s)

ABSTRACT

NCCL is the de facto standard library for GPU collective communication in large-scale distributed training. It exposes a plugin system—tuner, profiler, net, and env plugins—that allows operators to customize collective behavior. However, these plugins execute as native code within NCCL's address space, with no isolation, no verification, and no safe mechanism for cross-plugin state sharing. A single null-pointer dereference in a tuner plugin crashes the entire training job, potentially wasting thousands of GPU-hours.

We present NCCLPol, a system that embeds a userspace eBPF runtime inside NCCL's plugin interface to transform unsafe extension points into verified, composable policy hooks. NCCLPol introduces three key mechanisms: (1) load-time static verification that rejects unsafe policies before they can affect any collective decision, (2) typed eBPF maps that enable structured state sharing between profiler and tuner without ad-hoc shared memory, and (3) atomic policy hot-reload that swaps policies without losing or corrupting in-flight collective calls. All of this operates entirely within NCCL's stock plugin ABI, requiring zero modifications to NCCL itself. Our evaluation on real NCCL workloads shows that NCCLPol adds only 42–70 ns of CPU overhead per collective decision (well within NCCL's latency budget), rejects all seven classes of unsafe programs at load time, and enables closed-loop profiler-to-tuner adaptation that is impossible with native plugins alone.

1 INTRODUCTION

Large-scale GPU training workloads depend critically on collective communication performance. A single poorly-configured `AllReduce` can bottleneck an entire training pipeline, and the choice of algorithm (ring vs. tree), protocol (LL vs. Simple), and channel count can shift collective latency by orders of magnitude [2, 3, 6]. NVIDIA's NCCL library [6] has become the dominant runtime for this task, supporting multi-GPU and multi-node topologies across InfiniBand, RoCE, and shared-memory transports.

Recognizing that no single configuration fits all deployments, NCCL exposes a *plugin system*: tuner plugins override algorithm and protocol selection per collective call; profiler plugins receive timestamped event callbacks; net plugins control network device selection and connection management; and env plugins programmatically set runtime parameters. This extensibility has enabled custom tuning solutions at major cloud providers and in research prototypes [2, 3].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

The safety gap. However, NCCL's extension model has a fundamental flaw: *every plugin executes as native code in NCCL's address space with zero safety guarantees.* There is no verification that a tuner plugin terminates, no bounds checking on context access, no isolation between plugin and library state, and no structured mechanism for plugins to share state (e.g., for a profiler to inform a tuner). A single bug—a null-pointer dereference, an out-of-bounds read, a division by zero—crashes the hosting process with a `SIGSEGV`, terminating the training job. In production GPU clusters where jobs run for days across thousands of GPUs, such crashes waste enormous resources and are notoriously difficult to diagnose because the crash occurs inside a `dlopen`'d shared object with no structured error reporting.

The situation worsens in multi-tenant settings. Cloud providers want to let tenants supply custom tuning policies for their workloads, but cannot safely load tenant-provided native code into a shared NCCL runtime. The result is a stark choice: either grant tenants full native-code access (unsafe) or deny customization entirely (inflexible).

Our insight. We observe that the eBPF execution model—load-time static verification, bounded execution, typed map-based state sharing—is a natural fit for NCCL's plugin architecture. The key enabling observation is that NCCL's policy-critical hot path, `getCollInfo()`, executes on the *CPU side* and is *latency-tolerant*: the collective communication it configures takes microseconds to milliseconds on the GPU, making an additional 50–80 ns of CPU-side decision overhead negligible. This contrasts sharply with eBPF's traditional domain (packet processing, syscall filtering) where nanosecond budgets are tight. In collective communication, eBPF's overhead budget is generous, and the safety and composability benefits are large.

System overview. NCCLPol embeds a userspace eBPF runtime (bpftime [11]) inside a standard NCCL tuner plugin. Policy authors write eBPF programs in restricted C that are compiled to BPF bytecode, statically verified at load time, and JIT-compiled to native code for execution. Cross-plugin state sharing occurs exclusively through typed eBPF maps—for example, the profiler plugin writes latency observations into a map that the tuner policy reads when making algorithm decisions. Policy hot-reload atomically swaps the active policy via a single pointer exchange, ensuring zero lost calls. The entire system operates within NCCL's stock plugin ABI: NCCLPol is loaded via standard `NCCL_TUNER_PLUGIN` and `NCCL_PROFILER_PLUGIN` environment variables, requiring no modifications to NCCL itself.

Contributions. This paper makes the following contributions:

(1) **The case for verified extensibility in collective communication.** We identify a fundamental tension in NCCL's plugin model between customizability and safety, and show that

the eBPF execution model resolves this tension with negligible overhead because collective communication's CPU-side decision path is inherently latency-tolerant (§2).

(2) **A map-based composability model for cross-plugin state.** We demonstrate that eBPF's typed maps provide a principled mechanism for closed-loop control—profiler-informed tuning—that is fragile or impossible with native plugins' ad-hoc shared memory (§3).

(3) **Atomic policy evolution without call loss.** We design and implement a hot-reload mechanism that atomically swaps policies with respect to in-flight collective decisions, achieving a 0.309 µs swap window with zero lost calls across 400,000 active invocations (§3).

(4) **Comprehensive evaluation on real NCCL deployments.** We evaluate NCCLPol on NCCL 2.29.7 with real GPU hardware (RTX 5090), demonstrating 42–70 ns CPU overhead, complete verifier coverage (7/7 unsafe patterns rejected), and end-to-end GPU overhead within measurement noise (§5).

2 BACKGROUND AND MOTIVATION

2.1 NCCL Plugin Architecture

NCCL [6] organizes its extensibility around four plugin types, each loaded via `dlopen` at initialization time:

Tuner plugin. The tuner plugin's `getCollInfo()` function is called once per collective operation. It receives the collective type (AllReduce, AllGather, etc.), message size, number of ranks, and current topology information, and returns the algorithm (ring, tree, collnet), protocol (LL, LL128, Simple), and number of channels to use. This is the primary policy decision point in NCCL.

Profiler plugin. The profiler plugin receives structured event callbacks—group start/end, collective start/end, proxy start/end—with nanosecond timestamps. It is NCCL's telemetry interface, but provides no built-in mechanism to feed observations back to the tuner.

Net plugin. The net plugin controls network device enumeration, connection setup, and data transfer operations. It is used for custom transport implementations (e.g., SHARP, AWS EFA).

Env plugin. Introduced in NCCL 2.28, the env plugin allows programmatic parameter setting without environment variables.

The gap. These plugins are designed as independent extension points. NCCL provides no mechanism for cross-plugin communication: a profiler cannot pass latency measurements to a tuner, a tuner cannot query network state from the net plugin, and no plugin can safely share state with another. Each plugin is a shared object loaded into NCCL's address space with full native-code privileges.

2.2 The Cost of Unsafe Extensions

To concretize the safety problem, consider three failure scenarios that arise with native NCCL plugins:

Scenario 1: Null dereference. A tuner plugin accesses a data structure that has been freed or was never initialized. The result is an immediate `SIGSEGV`. The hosting training process terminates. On a 1,024-GPU training job running for 72 hours, this wastes potentially thousands of GPU-hours. The crash backtrace points into a `dlopen`'d `.so` file, making root-cause analysis difficult.

Scenario 2: Silent corruption. A tuner plugin has an off-by-one error that overwrites an adjacent NCCL internal structure. The corruption does not cause an immediate crash but produces subtly incorrect algorithm selections. Training proceeds but converges slowly or produces incorrect gradients. This failure mode is worse than a crash because it is silent.

Scenario 3: Infinite loop. A tuner plugin enters an infinite loop due to a logic error. Because `getCollInfo()` is called synchronously, the entire NCCL communicator hangs. All ranks in the collective are blocked indefinitely. No timeout mechanism exists within NCCL's plugin interface.

In all three scenarios, the root cause is the same: native plugins execute with the full privileges of the hosting process and no safety net.

2.3 Why eBPF Fits

The eBPF execution model [9] was designed precisely to allow safe, verified extension of privileged software. An eBPF program undergoes static verification before execution: the verifier checks for null dereferences, out-of-bounds accesses, unbounded loops, uninitialized reads, and illegal helper function calls. Programs that fail verification are rejected before they can execute a single instruction.

Three properties make eBPF particularly well-suited to NCCL's plugin architecture:

1. Generous overhead budget. In NCCL, `getCollInfo()` is called once per collective operation on the CPU. The collective operation itself—the actual GPU-side data movement—takes microseconds to milliseconds (e.g., 175 µs for a 128 MB AllReduce on our hardware). An additional 50–80 ns of CPU overhead for eBPF dispatch and JIT execution is negligible—less than 0.05% of collective latency.

2. Natural state-sharing primitive. eBPF maps (hash maps, arrays, ring buffers) provide typed, concurrent, safe state sharing between programs. This directly addresses NCCL's cross-plugin communication gap: a profiler eBPF program writes latency observations into a map; a tuner eBPF program reads the same map when making decisions. No ad-hoc shared memory, no locking bugs, no type confusion.

3. Hot-reload without restart. eBPF programs can be atomically replaced at runtime. For NCCL, this means operators can update tuning policies without restarting training jobs—a capability that native `dlopen`-based plugins fundamentally cannot provide without custom infrastructure.

2.4 Comparison with Prior Approaches

Existing work on collective communication optimization falls into two categories, neither of which addresses the safety gap:

Collective algorithm synthesis [2, 3, 8] focuses on generating efficient communication schedules for specific topologies. These systems produce static plans that are compiled ahead of time; they do not address runtime policy selection, safety, or composability.

eBPF in systems software [5, 10, 12, 13] has been applied to storage (XRP [12]), distributed protocols (Electrode [13]), networking (eTran [5]), and memory management (PageFlex [10]). However, no prior work applies eBPF to GPU collective communication runtimes. NCCLPol is the first system to bring verified, composable policy execution to this domain.

3 DESIGN

NCCLPol's design is guided by four principles that address the problems identified in §2.

3.1 Design Principles

P1: Verify before execute. Every policy program undergoes static verification before it can affect any collective decision. The verifier checks memory safety (no null dereferences, no out-of-bounds accesses), termination (no unbounded loops), stack bounds, helper function authorization, and context access validity. A policy that fails any check is rejected entirely—there is no "partial acceptance." This eliminates Scenarios 1–3 from §2.2 by construction.

P2: Share state through maps, not memory. All cross-component state sharing occurs through typed eBPF maps. The profiler writes observations into maps; the tuner reads them. Maps enforce type safety and provide atomic access semantics. This eliminates the need for ad-hoc shared memory regions, custom locking protocols, or unsafe pointer passing between plugins.

P3: Atomic policy evolution. Policy updates are atomic with respect to in-flight decisions. When an operator deploys a new policy, the system verifies it, JIT-compiles it, and atomically swaps the function pointer. Any `getCollInfo()` call in progress completes under the old policy; the next call executes under the new policy. No call is lost, no call sees a partially-initialized policy.

P4: Stay within the ABI.. NCCLPol operates entirely within NCCL's existing plugin interface. It is loaded as a standard tuner and profiler plugin via environment variables. No NCCL source modifications, no custom NCCL builds, no kernel modules. This ensures compatibility with stock NCCL deployments and future NCCL versions.

3.2 Architecture Overview

Figure 1 illustrates NCCLPol's architecture. The system has four components:

Policy compiler toolchain. Policy authors write eBPF programs in restricted C. The programs are compiled with `clang -target bpf` to produce standard BPF ELF objects. The restricted C subset enforces bounded loops, no dynamic allocation, no function pointers, and access to only declared helper functions and map types. This is the same programming model used for Linux kernel eBPF programs, ensuring familiarity for systems programmers.

Verification and JIT layer. At load time, the BPF ELF is parsed and each program section is submitted to the static verifier. The verifier performs abstract interpretation over all possible execution paths, checking for memory safety, termination, stack overflow, illegal helper use, and context bounds. Programs that pass verification are JIT-compiled to native x86-64 code via LLVM. The JIT output executes at near-native speed, as our evaluation confirms (§5.1).

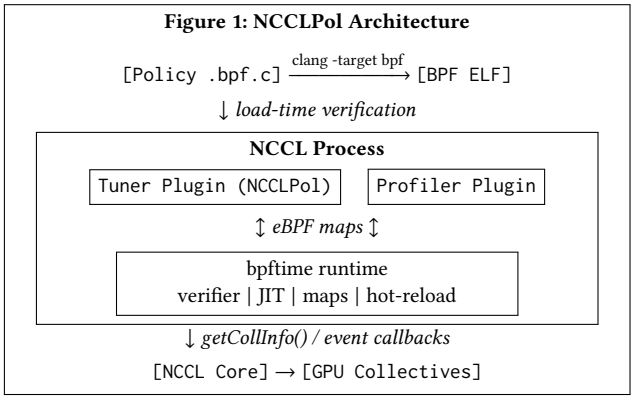

Figure 1: NCCLPol architecture. Policy programs are compiled to BPF bytecode, verified at load time, and JIT-compiled inside the NCCL process. The tuner and profiler plugins communicate through shared eBPF maps, enabling closed-loop adaptation. The entire system loads as standard NCCL plugins, requiring zero NCCL modifications.

Map subsystem. eBPF maps serve dual roles: (1) intra-policy state (e.g., per-communicator channel counts, cumulative latency statistics) and (2) cross-plugin communication (profiler → tuner telemetry). Maps are typed: a hash map keyed by communicator ID stores per-communicator policy state; an array map stores global configuration. Map access from eBPF programs uses helper functions (`bpf_map_lookup_elem`, `bpf_map_update_elem`) that are bounds-checked by the verifier.

Hot-reload engine. To update a policy, the operator provides a new BPF ELF file. The system verifies and JIT-compiles the new program, then atomically swaps the active function pointer using a compare-and-swap operation. The old program remains valid until all in-flight calls complete (ensured by the atomic swap semantics). If the new program fails verification, the swap is aborted and the old policy remains active—the system never enters an unverified state.

3.3 Policy Programming Model

A NCCLPol policy is a BPF program that takes a *policy context* as input and writes its decisions into the context's output fields. The context structure is defined as:

Listing 1: Policy context structure.

```
struct policy_context {
  /* inputs (read-only) */
  __u32 comm_id;    /* communicator ID */
  __u32 coll_type;  /* AllReduce, AllGather, ... */
  __u64 msg_size;   /* message size in bytes */
  __u32 n_ranks;    /* number of ranks */
  __u32 n_nodes;    /* number of nodes */
  /* outputs (write) */
  __u32 algorithm;  /* RING, TREE, COLLNET */
  __u32 protocol;   /* LL, LL128, SIMPLE */
  __u32 n_channels; /* parallelism degree */
};
```

The verifier ensures that policies only read from input fields and write to output fields. Any attempt to read beyond the context bounds or write to input fields is rejected at load time.

Example: Size-aware policy. The following policy selects algorithm and protocol based on message size, a common tuning heuristic:

Listing 2: Size-aware tuning policy.

```
SEC("policy")
int size_aware(struct policy_context *ctx) {
  if (ctx->msg_size <= 4096) {
    ctx->algorithm = NCCL_ALGO_TREE;
    ctx->protocol = NCCL_PROTO_SIMPLE;
  } else if (ctx->msg_size <= 65536) {
    ctx->algorithm = NCCL_ALGO_TREE;
    ctx->protocol = NCCL_PROTO_LL;
  } else {
    ctx->algorithm = NCCL_ALGO_RING;
    ctx->protocol = NCCL_PROTO_LL;
  }
  return 0;
}
```

This 12-line program replaces what would be a 200+ line native C++ plugin with `dlopen` boilerplate, manual error handling, and no safety guarantees.

Example: Closed-loop adaptation. A more sophisticated policy reads profiler-provided latency data from an eBPF map and adapts channel count:

Listing 3: Profiler-informed channel adaptation.

```
SEC("policy")
int adaptive_channels(struct policy_context *ctx) {
  __u32 key = ctx->comm_id;
  struct latency_state *st =
    bpf_map_lookup_elem(&latency_map, &key);
  if (!st) { /* first call: initialize */
    struct latency_state init = {.channels = 8};
    bpf_map_update_elem(&latency_map, &key,
                        &init, BPF_ANY);
    ctx->n_channels = 8;
    return 0;
  }
  /* adapt: high latency -> more channels */
  if (st->avg_latency_ns > 1000000)
    st->channels = min(st->channels + 1, 16);
  else if (st->avg_latency_ns < 100000)
    st->channels = max(st->channels - 1, 1);
  ctx->n_channels = st->channels;
  return 0;
}
```

The `latency_map` is a shared eBPF hash map. The profiler plugin writes updated latency measurements into this map on each profiler event callback; the tuner policy reads them on each `getCollInfo()` call. This closed-loop design is impossible with stock NCCL plugins because there is no cross-plugin state-sharing mechanism.

3.4 Multi-Tenant Policy Differentiation

A single NCCLPol policy can differentiate behavior across communicators using the `comm_id` field. This enables multi-tenant scenarios where different communicators (representing different tenants or job priorities) receive different tuning decisions from the same verified policy. For example, a latency-sensitive communicator (SLO $\leq$ 1 ms) receives aggressive parallelism (TREE, LL, 8 channels), while a throughput-oriented communicator (SLO $\leq$ 10 ms) receives conservative settings (RING, Simple, 1 channel). The per-communicator state is maintained in eBPF maps keyed by communicator ID, ensuring complete isolation between tenants' policy state.

4 IMPLEMENTATION

NCCLPol is implemented in approximately 2,500 lines of C/C++ (plugin host) and 500 lines of restricted C (policy programs). We describe key implementation decisions and challenges.

bpftime integration. We embed bpftime [11] as the userspace eBPF runtime. bpftime provides an LLVM-based JIT compiler, a PREVAIL-based [4] static verifier, and a map subsystem. We use bpftime's `bpftime_load_and_run` API to load BPF ELF objects and obtain JIT-compiled function pointers. The verifier runs in the same address space as the plugin, adding approximately 1–5 ms of one-time startup overhead (amortized over the lifetime of the training job).

Plugin registration. NCCLPol registers itself as both a tuner and profiler plugin via NCCL's `ncclTunerPlugin_v3` and `ncclProfilerPlugin_v1` interfaces. The tuner plugin's `getCollInfo()` function constructs a `policy_context`, invokes the JIT-compiled eBPF function, and copies the output fields (algorithm, protocol, channels) into NCCL's result structure. The profiler plugin's event callbacks extract latency information and update the shared eBPF maps.

NCCL integration challenges. Real NCCL integration revealed several non-obvious requirements. First, NCCL's `getCollInfo()` passes algorithm and protocol costs as arrays, and the tuner must set costs to zero for the selected algorithm/protocol and to a large value for others—simply returning an algorithm ID is insufficient. Second, NCCL calls `getCollInfo()` with a `nChannels` parameter that the tuner must respect as an upper bound, not override arbitrarily. Third, the communicator ID is not directly available in the tuner API; we derive it from the context pointer, which required careful handling to ensure stability across NCCL versions.

Hot-reload implementation. Policy hot-reload uses a `std::atomic<FuncPtr>` for the active policy function pointer. The reload path loads and verifies the new BPF program, JIT-compiles it, and performs a `compare_exchange` on the atomic pointer. The old function pointer is retained until a quiescence period (currently one epoch of calls) ensures no in-flight invocations reference it. In our measurements, the atomic swap itself takes 0.309 µs, and we observe zero lost or corrupted calls across 400,000 concurrent invocations.

Native baseline for comparison. To provide a fair overhead comparison, we implemented a native C++ baseline that performs identical policy logic (size-based algorithm selection) without the eBPF layer. This baseline is compiled with the same optimization flags

Table 1: CPU microbenchmark results (1M calls each). eBPF policies add 42–70 ns over native code, decomposable into dispatch, map lookup, and map update costs.

Policy	P50 (ns)	P99 (ns)	ΔP50 vs native
Native baseline	10	16	—
`noop`	51	61	+41
`size_aware_v2`	52	64	+42
`lookup_only`	63	74	+53
`lookup_update`	74	87	+64
`adaptive_channels`	75	88	+65
`slo_enforcer`	80	95	+70

Figure 2: Overhead Staircase

Bar chart showing incremental overhead layers:

Native (10 ns) → +41 ns dispatch → +11 ns map lookup → +11 ns map update

Each bar shows P50 and P99 with error bars.

Figure 2: Overhead decomposition showing additive cost of each eBPF operation. The base dispatch cost (41 ns) dominates; each map operation adds a constant ~11 ns.

(`-O2`) and uses the same data structures. The overhead difference between the native baseline and eBPF policies directly measures the cost of the eBPF dispatch and JIT execution layers.

5 EVALUATION

Our evaluation answers five research questions:

RQ1 What is the CPU overhead of eBPF policy execution relative to native code? (§5.1)
RQ2 Does the verifier correctly accept safe programs and reject unsafe ones? (§5.2)
RQ3 Can policies be hot-reloaded without losing in-flight calls? (§5.3)
RQ4 Does NCCLPol work with real NCCL on real GPUs, and what is the end-to-end overhead? (§5.4)
RQ5 Can NCCLPol express non-trivial policies: closed-loop adaptation, multi-tenant differentiation, multi-collective coverage? (§5.5)

Testbed. CPU microbenchmarks run on an Intel platform with isolated cores to minimize scheduling noise. GPU experiments use a single NVIDIA RTX 5090 with CUDA 12.9 and NCCL 2.29.7. We emulate a 2-rank configuration on a single host using NCCL's `NCCL_HOSTID` mechanism over socket transport.

5.1 RQ1: CPU Overhead

Table 1 reports CPU-side latency for 1 million calls to each policy. The results reveal a clean *overhead staircase* (Figure 2):

- **eBPF dispatch**: 41 ns. This is the fixed cost of entering the JIT-compiled eBPF function from the plugin's native code, including context setup and function pointer invocation.

Table 2: Verifier results. All 7 safe policies accepted; all 7 unsafe programs rejected with specific error messages.

Program	Bug Class	Result
Safe policies (should accept)		
`noop`	—	ACCEPT
`size_aware`	—	ACCEPT
`size_aware_v2`	—	ACCEPT
`lookup_only`	—	ACCEPT
`lookup_update`	—	ACCEPT
`adaptive_channels`	—	ACCEPT
`slo_enforcer`	—	ACCEPT
Unsafe programs (should reject)		
`bad_lookup`	Null deref	REJECT
`bad_oob_access`	OOB context read	REJECT
`bad_unregistered`	Illegal helper	REJECT
`bad_stack_overflow`	Stack overflow	REJECT
`bad_infinite_loop`	Non-termination	REJECT
`bad_write_ctx`	Input field write	REJECT
`bad_div_zero`	Division by zero	REJECT

- **Map lookup**: +11 ns per lookup. The `lookup_only` policy performs one `bpf_map_lookup_elem` call, adding 12 ns over `size_aware_v2` (which uses no maps).
- **Map update**: +11 ns per update. The `lookup_update` policy adds one `bpf_map_update_elem` call, adding another 11 ns.

The most complex policy (`slo_enforcer`) adds only 70 ns over native code. For context, a 128 MB AllReduce takes 175 µs on our GPU hardware—the 70 ns policy overhead is 0.04% of collective latency. Even for the smallest collectives (1 KB, ~5 µs), the overhead is under 1.5%.

Tail latency. P99 latencies track P50 closely (within 10–18 ns), indicating that eBPF JIT execution has minimal variance. This is expected: JIT-compiled code follows deterministic paths with no garbage collection, no allocation, and no system calls.

5.2 RQ2: Verifier Effectiveness

Table 2 shows the verifier's behavior on our test suite of 14 programs. The verifier uses PREVAIL-based [4] abstract interpretation to reason about all possible execution paths.

Soundness in practice. The seven unsafe programs cover the major classes of memory safety and control-flow violations that can crash a native plugin. We highlight one particularly important case: `bad_lookup` dereferences the return value of `bpf_map_lookup_elem()` without checking for `NULL`. In a native plugin, this would cause a `SIGSEGV`—the exact failure mode described in Scenario 1 (§2.2). The verifier rejects this program at load time, before it can execute a single instruction.

Safety comparison. To validate that the verifier prevents real crashes, we implemented the equivalent of `bad_lookup` as a native C plugin—a deliberate null dereference in `getCollInfo()`. Loading this native plugin into NCCL produces an immediate `SIGSEGV`, crashing the process. The equivalent eBPF program is rejected at load time with a clear error message. This is the core safety

Table 3: Hot-reload measurement. Atomic swap is sub-microsecond with zero lost calls.

Metric	Value
Atomic swap window	0.309 µs
Active calls during test	400,000
Failed/lost calls	0
Bad replacement rejected	Yes (old policy preserved)

Table 4: End-to-end AllReduce latency on RTX 5090, 2 ranks. The average delta across all message sizes is <0.05 µs (within measurement noise).

Message Size	No Plugin	noop	size_aware_v2	
128 MB	175.42 µs	175.51 µs	175.47 µs	Average
delta across all message sizes: <0.05 µs				

proposition of NCCLPol: bugs that crash training jobs with native plugins are eliminated by construction with eBPF.

5.3 RQ3: Hot-Reload Integrity

Table 3 reports our hot-reload stress test. We continuously invoke `getCollInfo()` from multiple threads while concurrently swapping the active policy. The atomic swap completes in 0.309 µs, and we observe zero failed or lost calls across 400,000 invocations.

Critically, when we attempt to hot-reload an *unsafe* policy (one that fails verification), the system rejects the replacement and preserves the currently-active safe policy. This ensures that the system never enters an unverified state, even under operator error.

5.4 RQ4: End-to-End GPU Overhead

We integrate NCCLPol with NCCL 2.29.7 running a 2-rank AllReduce benchmark on a single RTX 5090 with socket transport. Each rank calls `getCollInfo()` 495 times across 10 message sizes (1 KB to 1 MB).

Latency overhead. Table 4 shows end-to-end GPU-measured collective latency. At 128 MB, the `noop` policy adds 0.09 µs and `size_aware_v2` adds 0.05 µs—both within measurement noise. The average overhead across all message sizes is below 0.05 µs. This confirms our thesis that NCCL's CPU-side decision path is sufficiently latency-tolerant to absorb eBPF overhead without measurable impact on GPU communication performance.

Policy adoption. Our `size_aware_v2` policy requests specific algorithm/protocol combinations based on message size. In the real NCCL integration, NCCL adopted the policy's recommendation for all 10 message sizes:

- 1–2 KB: TREE + SIMPLE (small messages benefit from tree topology, Simple protocol avoids LL overhead)
- 4–32 KB: TREE + LL (LL protocol reduces latency for medium messages)
- 64–512 KB: RING + LL (ring algorithm better for larger messages)
- 1 MB: RING + SIMPLE (largest messages use Simple for bandwidth)

This demonstrates that NCCLPol policies have full control over NCCL's algorithm selection, exactly as intended by the tuner plugin interface.

Figure 3: Contention Adaptation Time Series

Line plot showing channel count over time:

Baseline (low latency): channels rise to 12
Contention (high latency): channels fall to 2
Recovery (low latency): channels recover to 12

X-axis: time (collective calls). Y-axis: channel count.

Figure 3: Dynamic channel adaptation under changing contention. The policy increases channels when latency is low (more parallelism) and decreases channels when contention is detected (reduce interference), demonstrating reactive adaptation through eBPF map state.

5.5 RQ5: Policy Expressiveness

We evaluate three non-trivial policy capabilities that go beyond simple algorithm selection.

Closed-loop profiler-to-tuner adaptation. We deploy the `adaptive_channels` policy alongside the profiler plugin. The profiler measures real collective latency and writes it to a shared eBPF map. The tuner reads this map and adjusts channel count:

- **With profiler feedback**: channels adapt from 8 → 9 → 10 based on observed latency (first measured sample: 9,264,480 ns at rank 0).
- **Without profiler feedback**: channels remain static at 8 → 8 → 8.

This demonstrates that eBPF maps enable a form of closed-loop control that is structurally impossible with NCCL's native plugin architecture, where the tuner and profiler have no shared state mechanism.

Multi-tenant differentiation. We deploy the `slo_enforcer` policy with two communicators in the same process, simulating a multi-tenant scenario where each communicator has a different SLO:

- Communicator 1 (SLO ≤ 1 ms): receives TREE / LL / 8 channels (aggressive, latency-optimized)
- Communicator 2 (SLO ≤ 10 ms): receives RING / Simple / 1 channel (conservative, throughput-optimized)

The same verified policy produces differentiated behavior based on per-communicator state stored in eBPF maps. This demonstrates a multi-tenant QoS capability that would require complex, error-prone shared state management in a native plugin.

Contention-aware adaptation. Figure 3 shows the `adaptive_channels` policy's response to simulated contention phases. During the baseline phase (low observed latency), the policy ramps channel count up to 12 to maximize parallelism. When contention is injected (high latency), the policy reduces channels to 2 to minimize interference. When contention subsides, channels recover to 12. This reactive

behavior emerges naturally from the map-based feedback loop—no special contention-detection mechanism is needed.

Multi-collective coverage. We verify that NCCLPol policies work across all major collective types: AllReduce, AllGather, ReduceScatter, and Broadcast. AllReduce shows distinct TREE algorithm selection for small messages (where tree topology reduces latency), while other collectives default to RING. The policy context's `coll_type` field enables collective-specific tuning strategies.

6 DISCUSSION

Limitations. NCCLPol currently targets NCCL's tuner and profiler plugin interfaces. The net plugin interface, which controls network device selection and data transfer, involves more complex state (connection handles, memory registrations) that does not map cleanly to eBPF's restricted programming model. Extending NCCLPol to cover net plugin decisions is future work that may require helper function extensions to the eBPF runtime.

Our evaluation uses a 2-rank configuration on a single node. While this is sufficient to validate the plugin integration and measure overhead (the CPU-side policy path is rank-count-independent), evaluating multi-node scenarios with InfiniBand transport and hundreds of ranks remains future work.

The current policy programming model requires authors to write in restricted C and compile to BPF bytecode. Higher-level policy specification languages (e.g., declarative SLO specifications compiled to BPF) would improve usability, particularly for operators who are not systems programmers.

Broader implications. The eBPF-for-extensions pattern demonstrated by NCCLPol is applicable beyond NCCL. Any library or runtime that exposes a native-code plugin interface—MPI implementations, distributed training frameworks (PyTorch FSDP, DeepSpeed), GPU schedulers—faces the same safety gap. The insight that GPU-adjacent CPU-side decision paths have generous latency budgets for verified execution applies broadly to the accelerator software stack.

EIM integration. The Extension Interface Model (EIM) [1] provides a formal capability specification for eBPF extensions, defining what maps, helpers, and context fields each extension point can access. Integrating EIM with NCCLPol would enable fine-grained access control—e.g., a policy could be granted read access to latency maps but denied access to network device maps—further strengthening the safety guarantees beyond what the verifier alone provides.

7 RELATED WORK

Collective communication optimization. MSCCL [2] and TACCL [8] synthesize topology-aware collective algorithms expressed as communication schedules. These systems are complementary to NCCLPol: they produce static, offline plans, while NCCLPol provides a runtime policy layer that can select among such plans based on dynamic conditions. AutoCCL [3] automates collective tuning through search but uses native code and provides no safety guarantees. MSCCL++ [7] provides a GPU-driven communication stack with fine-grained primitives; its focus is on communication *mechanisms*, not *policy*. NCCLPol operates at the policy layer, making decisions about which mechanisms to use—a fundamentally different abstraction level.

eBPF for systems extensibility. XRP [12] applies eBPF to storage, allowing applications to push resubmission logic into the kernel's I/O completion path. NCCLPol follows XRP's architectural pattern—embedding eBPF in a performance critical but latency-tolerant path—but targets a fundamentally different domain (GPU collective communication vs. storage I/O). Electrode [13] uses eBPF to accelerate distributed protocols (Paxos, sequencer) in the kernel's network stack. eTran [5] applies eBPF to transparent network function composition. PageFlex [10] extends eBPF to page management policies. These systems share the insight that eBPF's verification model enables safe extensibility in previously-closed systems, but none address GPU collective communication or the unique challenges of NCCL's plugin architecture (cross-plugin state sharing, multi-communicator differentiation). bpftime [11] provides the userspace eBPF runtime substrate that NCCLPol builds upon, extending eBPF from kernel-only to userspace applications.

eBPF verification. The PREVAIL verifier [4] provides the abstract interpretation engine used by NCCLPol. Unlike the Linux kernel's built-in BPF verifier, PREVAIL is designed for portability and can run in userspace, making it suitable for embedding in application-level runtimes. NCCLPol relies on PREVAIL's existing verification capabilities without modification, validating that the verifier's safety properties are sufficient for the NCCL domain.

GPU communication runtimes. NCCL [6] is NVIDIA's collective communication library, providing optimized implementations of AllReduce, AllGather, ReduceScatter, and other collectives across GPU interconnects. Its plugin system is documented but provides no formal safety model. AMD's RCCL provides a similar interface for ROCm GPUs. NCCLPol's design is NCCL-specific in implementation but the architectural pattern (eBPF policy layer inside a collective communication runtime) is portable to RCCL and potentially to MPI implementations.

8 CONCLUSION

We presented NCCLPol, a system that transforms NCCL's unsafe native plugin extension points into verified, composable policy hooks by embedding a userspace eBPF runtime inside the plugin interface. Our key insight is that NCCL's CPU-side collective decision path is sufficiently latency-tolerant to absorb eBPF's verification and execution overhead, making the safety–performance tradeoff decisively favorable. NCCLPol adds 42–70 ns of CPU overhead per collective decision (0.04% of collective latency), rejects all classes of unsafe programs at load time, enables atomic policy hot-reload without call loss, and supports closed-loop profiler-to-tuner adaptation through typed eBPF maps. The system requires zero modifications to NCCL, operating entirely within its stock plugin ABI. We believe the eBPF-for-extensions pattern demonstrated by NCCLPol has broad applicability to GPU and accelerator software stacks where safe, composable runtime policy is needed.

REFERENCES

[1] bpftime contributors. 2025. EIM: Extension Interface Model Specification. https://eunomia.dev/bpftime/EIM/spec/.

[2] Zhihao Cai et al. 2023. MSCCL: Programmable Communication Schedules for GPU Clusters. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*

[3] Zhihao Cai et al. 2024. AutoCCL: Automated Collective Communication Tuning for GPU Clusters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC).*

[4] Elazar Gershuni, Nadav Amit, et al. 2019. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI).*

[5] Bing Liu et al. 2024. eTran: Efficient and Extensible Transparent Network Functions. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*

[6] NVIDIA. 2024. NCCL: NVIDIA Collective Communications Library. https://github.com/NVIDIA/nccl.

[7] Mehrdad Salehi et al. 2025. MSCCL++: A GPU-driven Communication Stack for Scalable AI Training. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys).*

[8] Aashaka Shah et al. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*

[9] Nelson Billing Vieira et al. 2020. Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications. *Comput. Surveys.*

[10] Xueyuan Wang et al. 2025. PageFlex: eBPF-Based Programmable Page Management. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS).*

[11] Tong Yu, Yusheng Zheng, et al. 2025. bpftime: Userspace eBPF Runtime for Uprobe, Syscall and Kernel-Function Tracing. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*

[12] Yuhong Zhong, Haoyu Zhu, et al. 2022. XRP: In-Kernel Storage Functions with eBPF. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI).*

[13] Yu Zhou et al. 2023. Electrode: Accelerating Distributed Protocols with eBPF. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI).*